Computer Engineering Design Proposal

1.1 Introduction

Decision on how to structure operations core of the SSTV decoder on the rp2350 MCU. Should bare metal programs be used, vs RTOS.

1.2 Objectives

- Evaluate interrupt profile of the application IO. How many interrupts will probably be needed?
- Determine if FreeRTOS can run on the purdue proton.
- Evaluate how much extensibility should be prioritized for the project. Eg, easily modifiable code?
- Evaluate any efficiency penalties associated with RTOS. Is it too much memory? Too slow?
- Make a determination of whether an RTOS or bare metal implementation would be superior

---

2. Design Requirements

- **Functional Requirements:** Need efficient way to schedule tasks and respond to interrupts for many IO peripherals, and between processors

- **Non-Functional Requirements:** Real time performance, maximize cpu usage, extensibility

- **Constraints:** available memory, power (if battery), overhead

---

3. Candidate Architectures / Design Approaches

3.1 Bare Metal

- **Description:** Run all code directly on the microprocessor, in the style used in labs. Handle interrupts manually, and inter process communication between 2 cores. Manual orchestration of tasks

- **Block Diagram (if applicable).**

- **Key Components:** Interrupt service routines, FIFO mailboxes, superloops

- **Pros:** Simplicity, familiarity compared to labs, more experience, much lower memory footprint

- **Cons:** Exponential complexity, harder to debug, less modular, more difficult process coordination

- **Feasibility:** Feasible. All tools are available and knowledge is currently being harvested in class

3.2 RTOS Layer

- **Description:** Add a layer of abstraction with a lightweight RTOS (realtime operating system) to help manage interrupts, task scheduling, and process management

- **Block Diagram (if applicable).**

- **Key Components:** RTOS, task scheduler

- **Pros:** Greater resistance to change, easier debugging, predictable performance, well tested and proven IPC, easier to integrate new code

- **Cons:** More complicated, steeper learning curve, significantly higher memory cost, possibly significant overhead

Feasibility:

---

4. Comparative Analysis

A summary comparison of the candidate designs.

4.1 Decision Matrix (optional but recommended)

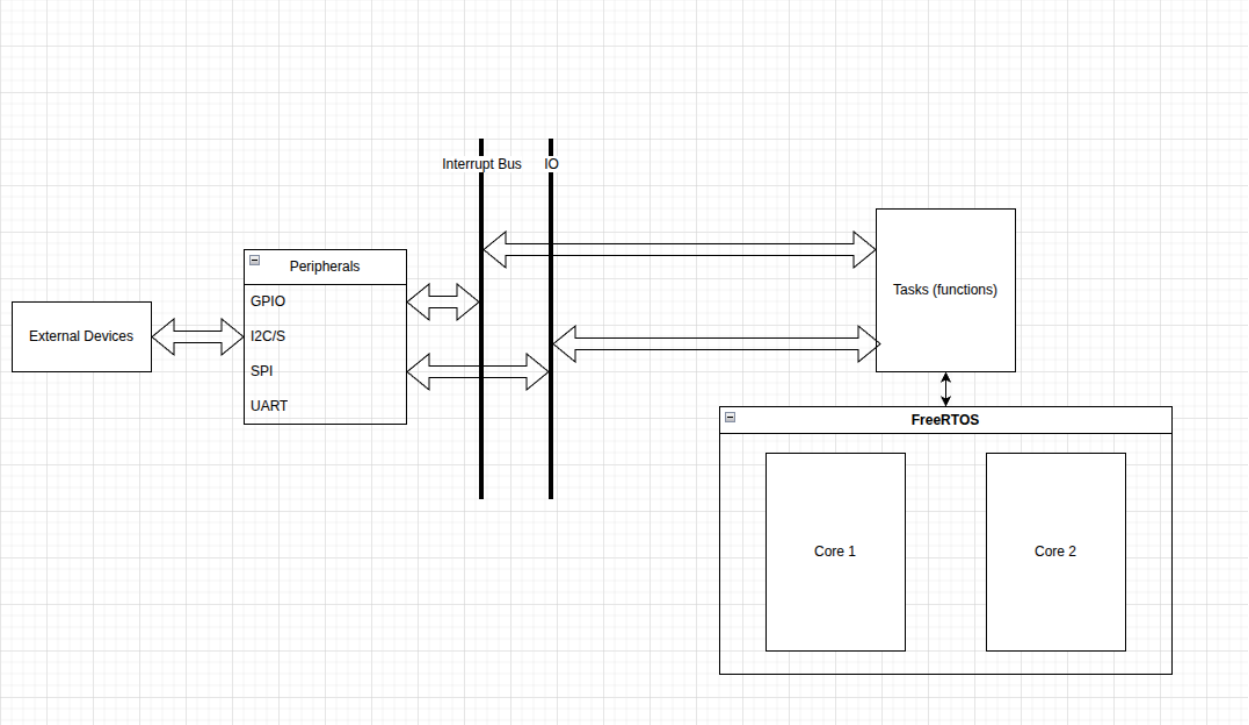
Criteria	Weight	Bare metal	RTOS
Performance	0.3	High	High
Power Efficiency	0.2	High	Medium
Time Cost	0.2	High	Medium
Ease of Implementation	0.3	Low	High
Total Score		0.5	0.8

---

5. Selected Architecture

- **Chosen Design:** RTOS

- **Rationale:** Ease of integration, improved integrity for multicore operations, large existing software ecosystem



- **RTOS interface Diagram**

---

6. Implementation Plan

- **Milestones & Timeline**
    1. Testing deployment of freeRTOS on the proton
    2. Building out the framework for input output and process queueing
- Would like to have a working prototype scheduling system by early october (5th of october)

---

7. Risk Analysis & Mitigation

- Potential risks
    - FreeRTOS turns out not to be compatible
    - Turns out resource usage it too high
    - Not allowed for project

- Mitigation strategies.
    - Maintain discipline to test all aspects of FreeRTOS deployment before committing
    - Retain open mind to switching back to bare metal

---

9. References

https://www.freertos.org/Documentation/01-FreeRTOS-quick-start/01-Beginners-guide/00-Overview